# Fiche express — oral Miaoumi

*L'essentiel en 2 pages, dans l'ordre de révision. Le détail est dans FICHE-REVISION-MIAOUMI.pdf. Oral demandé : 1) projet fonctionnel et finalité ; 2) maîtrise technique du code.*

## 1. La finalité — à dire en 1 minute

« Miaoumi est une plateforme de garde d'animaux entre particuliers. Un **propriétaire** publie une annonce (animal, ville, dates) ; des **gardiens** la trouvent grâce aux filtres et postulent. Le propriétaire accepte un gardien : l'annonce passe en « pourvue » et les autres candidatures sont refusées. À la fin, il clôture la garde, puis chacun note l'autre ; la note moyenne apparaît sur le profil. Un même compte peut être propriétaire et gardien. Techniquement : PHP sans framework en MVC, MySQL avec PDO, et une attention particulière à la sécurité. »

## 2. Le MVC et le chemin d'une requête

- **Modèle** = SQL + règles (valide avant d'écrire) · **Vue** = HTML, pas de SQL, tout passe par e() · **Contrôleur** = lit la requête, appelle le modèle, affiche ou redirige.
- Tout passe par **public/index.php** (front controller) → **bootstrap** : session, classes, e(), contrôle du jeton CSRF → **routeur** : ?page= → méthode du contrôleur.
- Page inconnue → 404 « introuvable » · MySQL éteint → 503 « indisponible ».

## 3. La base de données

- 4 tables : utilisateurs, annonces, candidatures, avis · 6 associations, toutes 0,n — 1,1.
- **Règle MCD → MLD** : la clé du côté 0,n descend en clé étrangère dans la table du côté 1,1.
- Clés étrangères : annonces.id_proprietaire · candidatures.id_annonce, id_gardien · avis.id_annonce, id_auteur, id_cible.
- Deux liens utilisateurs–avis (rédiger / recevoir) pour distinguer l'auteur de la personne notée ; leurs noms viennent des **rôles** Looping.
- Statuts : annonce ouverte → pourvue → terminee · candidature en_attente → acceptee / refusee · ON DELETE CASCADE.

## 4. La sécurité — 5 réponses à savoir par cœur

| Attaque | Réponse |
|---|---|
| XSS | Tout texte affiché passe par e() = htmlspecialchars : un <script> s'affiche au lieu de s'exécuter. |
| Injection SQL | Requêtes préparées PDO : prepare() avec des ?, puis execute() ; la saisie ne peut pas changer la requête. |
| CSRF | Jeton aléatoire en session, caché dans chaque formulaire, comparé avec hash_equals() à chaque POST ; sinon 403. |
| Mots de passe | password_hash() / password_verify(), jamais en clair ; même message si e-mail ou mot de passe faux. |
| Usurpation | L'auteur, le propriétaire, le gardien viennent de la session, jamais du formulaire ; les modèles revérifient le propriétaire. |

## 5. Une phrase par méthode (commence par ce que tu n'as pas codé)

| Méthode | Phrase |
|---|---|
| Candidature::postuler() | Transaction + verrou FOR UPDATE ; refuse annonce fermée, sa propre annonce ou doublon ; insère en en_attente. |
| Candidature::decider() | Seul le propriétaire, seulement si en_attente ; passe en acceptee ou refusee *(+ pourvue et refus des autres : à venir)*. |
| Candidature::pourUtilisateur() | Candidatures envoyées et reçues, avec les noms (jointures). |
| Avis::destinataires() | Après la date de fin : propriétaire ↔ gardiens acceptés, sans avis déjà déposé. |
| Avis::creer() | Transaction + verrou ; note 1–5, commentaire ≤ 2 000, cible autorisée ; insère. |
| Avis::pourAnnonce() | Avis d'une annonce avec noms de l'auteur et de la cible (deux jointures, alias). |
| Avis::noteMoyenne() | *À venir* : AVG des notes reçues, null s'il n'y a pas d'avis. |

| Méthode | Phrase |
|---|---|
| Utilisateur::creer() / trouverParEmail() | Insère avec password_hash() / renvoie id, nom, hash pour un e-mail. |
| Annonce::creer() | Valide champs et dates, renvoie un tableau d'erreurs ; vide = insérée. |
| Annonce::rechercher() | SELECT construit selon les filtres avec des ? ; sans propriétaire, seulement les ouvertes. |
| Annonce::cloturer() | Seul le propriétaire, pas déjà terminée ; passe en terminee (le WHERE répète les conditions). |
| Annonce::marquerPourvue() | Passe une annonce ouverte en pourvue ; true si une ligne a changé. |

## 6. Le socle commun

- **Gabarit** : chaque vue charge header.php (CSS, navigation, flash) et footer.php → même style partout.
- **Flash** : ajouterFlash() avant la redirection, lireFlash() lit puis efface → affiché une seule fois.
- **Post/Redirect/Get** : succès → redirection 303 ; rafraîchir ne renvoie pas le formulaire.
- **Erreur** : pas de redirection, la vue est réaffichée avec la saisie (sauf mots de passe) ; code 422.

## 7. Codes HTTP

303 redirection après succès · 403 interdit (CSRF, pas le droit) · 404 introuvable · 405 mauvaise méthode (GET au lieu de POST) · 409 doublon · 422 formulaire invalide · 503 base indisponible.

## 8. La démo — le parcours à savoir faire sans hésiter

1) Le propriétaire publie → 2) le gardien filtre et postule → 3) le propriétaire valide → 4) annonce pourvue, autre candidature refusée → 5) « Terminer la garde » (date_fin passée en base) → 6) chacun note l'autre, note moyenne sur le profil. Comptes de démo : sql/donnees-demo.sql, mot de passe **demo1234**.

## 9. Auto-test (fiche fermée, à voix haute, 30 s par question)

[ ] 1. Que se passe-t-il avec ?page=nimportequoi ?
[ ] 2. Pourquoi valider dans le modèle ?
[ ] 3. Pourquoi une redirection 303 ?
[ ] 4. Comment fonctionne le jeton CSRF ?
[ ] 5. À quoi sert FOR UPDATE dans postuler() ?
[ ] 6. Qui peut laisser un avis, et quand ?
[ ] 7. D'où viennent id_auteur et id_cible ?
[ ] 8. Pourquoi le propriétaire vient-il de la session ?